

Implementation Summary – WBOCRM

Project: Web-Based Operational CRM

Course: Istanbul Arel University – HW5 Part 2

Date: May 2026

Completed Phases

Phase 1 – Project Setup

- Monorepo structure: backend/ (Node.js) and frontend/ (React/Vite)
- Git repository initialised, .gitignore configured
- backend/package.json with all production and dev dependencies
- frontend/package.json with React, React Router, Axios, Vite

Phase 2 – Database Layer

- PostgreSQL schema in backend/src/db/schema.sql
- Tables: UserAccount, Lead, InteractionLog, SupportTicket
- pool.js with SSL auto-detection for Render deployment
- init.js (schema creation) and seed.js (demo data with bcrypt-hashed passwords)

Phase 3 – Backend API

- 3-tier architecture: Controller → Service → Repository
- Auth: POST /api/auth/login, POST /api/auth/register
- Leads: full CRUD + GET sorted by score + CSV export
- Tickets: full CRUD with optimistic locking
- Interaction logs: GET + POST per lead
- Dashboard: active leads, open tickets, top 5 leads, monthly revenue
- Users: GET all, PUT role (admin only)

Phase 4 – Middleware & Security

- authMiddleware.js: verifies JWT, attaches req.user
- rbacMiddleware.js: allowRoles(...roles) factory — returns 403 if role insufficient
- Input validation via express-validator on auth routes
- All SQL uses parameterised queries

Phase 5 – Lead Scoring Algorithm

- scoringService.js: 5-metric weighted scoring (0–100)
- Metrics: calls (×25), meetings (×25), budget (×20), companySize (×20), emailOpens (×10)
- Score calculated at lead creation time and stored in DB

Phase 6 – Frontend

- AuthContext with JWT decode, login/logout, persisted in localStorage
- api.js Axios instance: auto-injects Authorization: Bearer header
- Protected routing: ProtectedLayout + RoleRoute in App.jsx
- Pages:
- LoginPage — form with user_email / user_password fields
- DashboardPage — 4 KPI cards with 2-second auto-refresh

- LeadPage — table with create/edit/delete/profile navigation + CSV export button
- LeadProfilePage — unified customer profile: lead info + interaction timeline + linked tickets + admin assignment
- TicketPage — ticket list with create/update (optimistic locking conflict detection)
- UsersPage — admin-only user management with role dropdown
- Components: NavBar, LeadForm, TicketForm, PriorityTable
- Responsive layout, minimum viewport 360px

Phase 7 – Testing

- **38 unit tests** (Jest + mocked repositories): authService (6), scoringService (6), ticketService (6), leadService (12), dashboardController (6)
- **15 functional tests** (Jest + Supertest, live DB): lead CRUD, interaction logging, pipeline stage, tickets, dashboard KPIs, CSV export, PII masking, auth 401/403
- **20 BDD scenarios** (Cucumber): lead registration, duplicate email, ticket creation, auth flows, scoring, interaction logging, pipeline stage, dashboard KPIs, CSV export
- All tests passing; results saved to backend/test_results/

Phase 8 – Deployment Configuration

- render.yaml: Blueprint defining backend + frontend + PostgreSQL services
- backend/.env.example and frontend/.env.example
- deployment_info.txt: step-by-step deployment guide

Phase 9 – Documentation

- docs/architecture_description.md: 3-tier diagram, UML deployment/component diagrams, design decisions, schema
- docs/user_stories.md: 20 user stories across 7 epics with READ references
- docs/acceptance_tests.md: BDD scenarios mapped to user stories and NFRs
- docs/implementation_summary.md: this file
- README.md: full technical documentation for public repository

Phase 10 – GDPR/KVKK Compliance

- PII masking in leadController.js for support role (maskEmail, maskName functions)
- DELETE /api/leads/:id/personal-data — anonymises lead PII and deletes interaction logs
- DELETE /api/users/:id/personal-data — anonymises user account PII
- Admin self-erasure prevention and last-admin demotion guard in userController.js

Phase 11 – Ticket Archiving

- archiveService.js: daily cron job (setInterval 24h) moves resolved/closed tickets older than 365 days to ArchivedTicket table
- ArchivedTicket table added to schema.sql
- POST /api/tickets/archive manual trigger endpoint (admin only)
- Transactional INSERT-then-DELETE within a PostgreSQL transaction (BEGIN/COMMIT/ROLLBACK)

Phase 12 – Session Security & Offline Resilience

- 30-minute idle timeout in AuthContext.jsx with activity event listeners (mousemove, mousedown, keydown, scroll, touchstart)
- Offline ticket draft caching in TicketPage.jsx via localStorage (crm_pending_ticket) with Retry/Dismiss UI
- RoleRoute component in App.jsx for declarative RBAC route protection

Phase 13 – Demo Data Seeding

- POST /api/setup endpoint for one-time demo data seeding (3 users, 5 leads, 2 tickets)

- Accessible via API without CLI access — alternative to running `seed.js` manually

Technology Stack

Layer	Technology	Version
Backend runtime	Node.js	20 LTS
Backend framework	Express	4.18
Database driver	pg (node-postgres)	8.11
Auth	jsonwebtoken	9.x
Password hashing	bcryptjs	2.4
Validation	express-validator	7.x
Frontend framework	React	18.x
Build tool	Vite	5.x
Routing	React Router	v6
HTTP client	Axios	1.x
Unit/functional tests	Jest + Supertest	29.x / 6.x
BDD tests	@cucumber/cucumber	10.x
Database	PostgreSQL	15 (Render)
Deployment	Render	—

READ Compliance

Section	Requirement	Status
FR-AUTH-01/02	JWT login / 401 on invalid	■
FR-AUTH-03/04	401 no token / 403 wrong role	■
FR-LM-01 – FR-LM-05	Lead CRUD	■
FR-LM-06	Interaction log notes	■
FR-SC-01	Priority score 0–100	■
FR-SC-06	Lead assignment to sales rep	■
FR-ST-01/02	Ticket create / update	■
FR-DA-01	Dashboard KPIs	■
FR-DOC-15	CSV export	■
FR-UM-01/02	User listing / role update	■
NFR-ST-05	Optimistic locking	■
NFR-ST-07	GDPR/KVKK PII masking and erasure	■
NFR-ST-12	Dashboard 2s auto-refresh	■
NFR-ST-14	Offline ticket draft caching	■
NFR-ST-15	Ticket archiving (365+ days)	■
NFR-SEC-04	Idle session timeout (30 min)	■
Section 3.8.3	Unified Customer Profile	■
Section 14	BDD acceptance tests	■

Known Constraints

- Free Render tier: services spin down after 15 min inactivity (30s cold start)
- JWT expiry is 30 minutes; no refresh token mechanism (scope per READ)
- Scoring recalculated only at creation — editing metrics after creation does not update score (by design)
- Monthly revenue uses `created_at` as the date proxy for Closed leads